Danmarks Tekniske Universitet

02322 Machine Oriented Programming, Group 23

Authors

Aisha Farah Mohamud - s235123
Carl Johann Creutzberg - s245635
Simon Knudsen - s255785
Sarah Mikkelsen Dimovski - s235087

March 25, 2026

Table 1: Contributions to Assignments

	Aisha	Sarah	Simon	Carl
Assignment 1	0%	0%	50%	50%
Assignment 2	50%	50%	0%	0%
Assignment 3	0%	0%	50%	50%
Assignment 4	50%	50%	0%	0%
Assignment 5	25%	25%	25%	25%

Contents

1 Assignment 1 **1**
1.1 Flow diagram 1
1.2 Implementation of assignment 1 1

2 Assignment 2 **2**
2.1 Flow diagram 2 2
2.2 Implmenetation of assignment 2 3

3 Assignment 3 **4**
3.1 Flow diagram 3 4
3.2 Implementation of assignment 3 5

4 Assignment 4 **6**
4.1 Flow diagram 4 6
4.2 Implementation of assignment 4 6

5 Assignment 5 **7**
5.1 Flow diagram 5 7
5.2 Implementation 7

1 Assignment 1

The goal of the program is to find the midpoint between two even numbers A and B. The program increases A and decreases B step by step until they become equal and the result will be the midpoint of the two even numbers. Before solving assignment 1, we first created an overview of the problem by breaking it down into smaller parts. To do this, we used a flow diagram, which helped us understand the logic step by step before writing any code.

1.1 Flow diagram

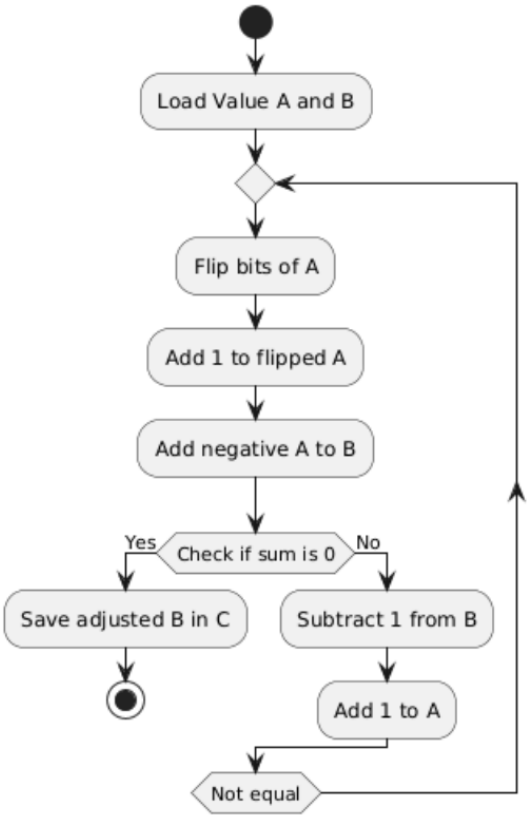

Figure 1: The flow diagram shows how A and B are adjusted until they meet in the middle

1.2 Implementation of assignment 1

The program starts by loading two values, A and B, into registers. A is negated by flipping its bits and adding one, which is how two's complement works. After that, it adds the negative version of A and B so its checking the difference between them. If the result is 0, it means A and B are equal and the loop will stop. If not, it will decrease B by 1 and increase A by 1 so both values move closer to each other. This will be repeated until they meet in the middle. Once they are equal, the program stores that value in C and then stops.

root/logo.pdf

2 Assignment 2

The goal of this assignment is to implement a function that reads a two digit decimal number from the console and returns the value in register R0. We created a flow diagram to understand the steps and structure of the program.

2.1 Flow diagram 2

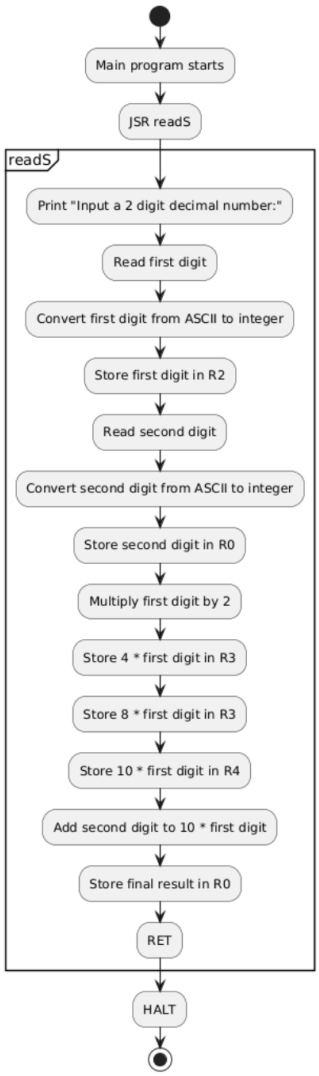

Figure 2: This flow diagram shows how the program reads two digits, converts them, combines them into one number, and stores the result in R0.

root/logo.pdf

2.2 Implmenetation of assignment 2

The program starts by calling the subroutine using JSR, which sends the program to readS where the main logic happens. The subroutine is called using JSR to ensure correct control flow and avoid access violations, since LC3 does not automatically manage function calls unless they are explicitly implemented. By using JSR, the program saves the return address and ensures that control can safely return to the main program after the subroutine has finished executing.

Inside the subroutine, the program first prints a message asking the user to input a two digit decimal number. W It then reads the first digit, converts it from ASCII to its actual numerical value by subtracting 48, and stores it in register R2.

After that, it reads the second digit, converts it in the same way, and stores it in register R0. Next, the program calculates 10 times the first digit. Since LC3 does not have a multiplication instruction, this is done step by step using repeated addition. The value is first doubled, then used to build up 4 times and 8 times the value, and finally combined to get 10 times the first digit. The second digit is then added to this value, so the full two digit number is formed and stored in register R0.

3 Assignment 3

The goal of the program is to check if a given number is prime and return 1 if it is, otherwise 0.

3.1 Flow diagram 3

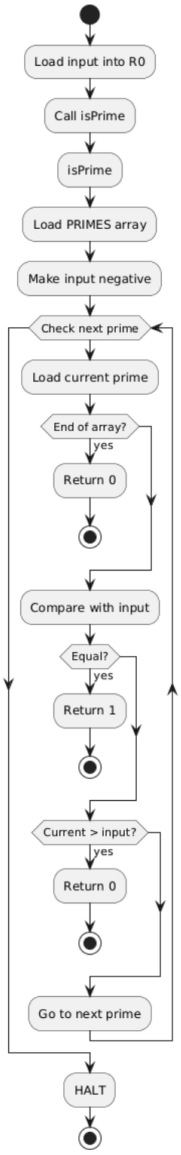

Figure 3: This flow diagram shows how the program checks if the input is a prime number and returns the result.

root/logo.pdf

3.2 Implementation of assignment 3

The program starts by calling the subroutine isPrime using JSR. The value to be checked is first loaded into register R0, which is used both as input and for returning the result.

In the subroutine, the program begins by loading the address of the array containing all prime numbers into register R5. This array is used to compare the input value against known prime numbers. The input value from R0 is then copied into register R3, and its twoâs complement is computed to create the negative version of the input. The program then enters a loop where it loads one prime number at a time from the array into register R2. If the value loaded is zero, it indicates that the end of the array has been reached and the number is therefore not prime. Otherwise, the program compares the current prime number with the input by adding the negative input to the current prime.

If the result of this comparison is zero, it means the input matches a prime number, and the program sets R0 to 1 before returning. If the result is positive, it means the current prime number is larger than the input, so the input cannot be found in the array and is not prime. In that case, R0 is set to 0 and the function returns.

If neither condition is met, the program moves to the next element in the array and repeats the process until a result is found.

4 Assignment 4

The goal is to implement a function resultS that takes a value in R0 and prints whether the number is prime or not based on whether the value is zero or non zero.

4.1 Flow diagram 4

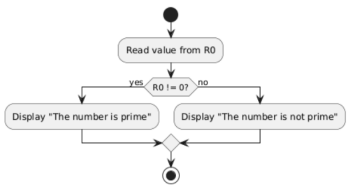

Figure 4: Flow diagram for assignment 4).

4.2 Implementation of assignment 4

The program starts by loading a value into register R0 and then calling the subroutine resultS using JSR, which transfers control to the function where the message is decided and printed. The value in R0 represents the result from a previous computation, where 1 means prime and 0 means not prime.

Inside the subroutine, the program first uses an ADD instruction on R0 to update the condition codes based on its value. This is done so the program can branch correctly depending on whether the value is zero or not. If the value in R0 is zero, the program branches to the label NOTPRIME, where it loads the message indicating that the number is not prime and prints it using PUTS.

If the value in R0 is not zero, the program continues and loads the message that the number is prime into R0 and prints it using PUTS. After printing the correct message in both cases, the function returns using RET, which ensures that control goes back to the main program, where the program then stops using HALT.

root/logo.pdf

5 Assignment 5

The goal of the program is to read a 2 digit number, determine whether it is prime, and display the result in a continuous loop.

5.1 Flow diagram 5

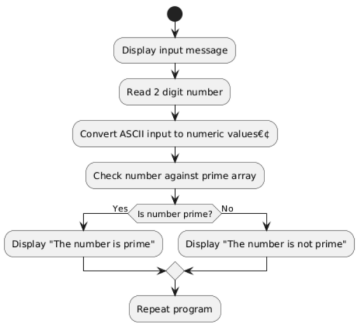

Figure 5: This flow diagram shows how the program reads a number, checks if it is prime, and displays the result in a loop

5.2 Implementation

The program is divided into three functions, readS, isPrime, and resultS, with START controlling the flow.

START prints the input message, calls the three functions in sequence, and then loops back. This ensures the program runs continuously and avoids placing the loop inside a subroutine.

readS reads two ASCII digits, converts them to integers by subtracting 48, and constructs the final number. Since LC3 has no multiplication, the first digit is multiplied by 10 using additions before adding the second digit.

isPrime checks if the number is in a predefined list of prime numbers. It loops through 25 values using a counter. If a match is found, it returns 1 in R0, otherwise 0. The function always ends with RET, which ensures proper return and avoids stack issues.

resultS prints the result based on R0. If R0 is zero, it prints not prime, otherwise it prints prime.